

IAT 461 / 882 — Assignment 2: Modeling Phigma User Behavior

Name: Tim Kung SFU ID: 301420651 Date: June 24

Before you start: read the full assignment README on Canvas.
Run cells top-to-bottom. Do **not** skip cells — later cells depend on earlier ones.
Every cell marked # YOUR CODE HERE must be completed.
Every cell marked `** Your interpretation: **` must be filled in with your own words.

Setup

Run this cell first. It imports all libraries used throughout the notebook.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns
import scipy.stats as stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
import warnings
from sklearn.preprocessing import MinMaxScaler, StandardScaler,
RobustScaler, OrdinalEncoder
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, classification_report,
    roc_auc_score, roc_curve, ConfusionMatrixDisplay
)
from sklearn.preprocessing import StandardScaler

# Plot style
plt.rcParams.update({
    'figure.dpi': 120,
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.titlesize': 13,
    'axes.labelsize': 11,
})
```

```
ACCENT = '#e8925a'    # course accent colour

print("Libraries loaded.")

Libraries loaded.
```

Part A — Linear Regression: Did the AI Assistant Change User Behavior?

Dataset: `phigma_ab_study.csv`

Research question: Does access to Pigma's AI design assistant increase whiteboard session duration among free-tier users?

This is a **between-subjects experiment**. Users were randomly assigned to treatment (AI access) or control (no AI). We use OLS regression to estimate the effect of the treatment while controlling for other variables.

No train/test split in this part. We are doing statistical inference — explaining the outcome — not predicting future users.

Load the data

```
df_ab = pd.read_csv('phigma_ab_study.csv', keep_default_na=False)

print(f"Shape: {df_ab.shape}")
print(f"\nColumn types:\n{df_ab.dtypes}")
print(f"\nFirst 3 rows:")
df_ab.head(3)
```

Shape: (2000, 12)

Column types:

user_id	str
ab_group	str
device_type	str
account_age_days	int64
prior_whiteboard_sessions	int64
plan_region	str
num_collaborators	int64
feature_tour_completed	int64
templates_used	int64
ai_suggestions_used	int64
whiteboard_actions_per_session	int64
avg_whiteboard_session_duration_min	float64
dtype:	object

First 3 rows:

	user_id	ab_group	device_type	account_age_days
prior_whiteboard_sessions \				
0	U76582	control	tablet	478
4				
1	U49158	control	desktop	633
9				
2	U82668	control	desktop	159
10				

	plan_region	num_collaborators	feature_tour_completed
templates_used \			
0	EU	3	1
0			
1	APAC	1	0
1			
2	NA	5	1
1			

	ai_suggestions_used	whiteboard_actions_per_session \
0	0	67
1	0	72
2	0	38

	avg_whiteboard_session_duration_min
0	29.70
1	32.32
2	17.91

Summary statistics

df_ab.describe(include='all').round(2)

	user_id	ab_group	device_type	account_age_days \
count	2000	2000	2000	2000.00
unique	1978	2	3	NaN
top	U21640	control	desktop	NaN
freq	2	1011	1211	NaN
mean	NaN	NaN	NaN	356.43
std	NaN	NaN	NaN	207.82
min	NaN	NaN	NaN	7.00
25%	NaN	NaN	NaN	178.00
50%	NaN	NaN	NaN	352.50
75%	NaN	NaN	NaN	533.25
max	NaN	NaN	NaN	729.00

	prior_whiteboard_sessions	plan_region	num_collaborators \
count	2000.00	2000	2000.00
unique	NaN	4	NaN
top	NaN	NA	NaN
freq	NaN	677	NaN
mean	4.51	NaN	2.16

std	3.33	NaN	2.37
min	0.00	NaN	0.00
25%	2.00	NaN	0.00
50%	4.00	NaN	1.00
75%	6.00	NaN	3.00
max	22.00	NaN	15.00

	feature_tour_completed	templates_used	ai_suggestions_used \
count	2000.00	2000.00	2000.00
unique	NaN	NaN	NaN
top	NaN	NaN	NaN
freq	NaN	NaN	NaN
mean	0.58	2.79	3.63
std	0.49	2.33	4.87
min	0.00	0.00	0.00
25%	0.00	1.00	0.00
50%	1.00	2.00	0.00
75%	1.00	4.00	7.00
max	1.00	16.00	29.00

whiteboard_actions_per_session	
avg_whiteboard_session_duration_min	
count	2000.00
2000.00	
unique	NaN
NaN	
top	NaN
NaN	
freq	NaN
NaN	
mean	84.55
39.53	
std	44.35
20.18	
min	8.00
6.27	
25%	54.00
25.51	
50%	76.00
35.40	
75%	106.00
48.93	
max	333.00
153.66	

Missing values

df_ab.isnull().sum()

user_id	0
ab_group	0

```

device_type          0
account_age_days     0
prior_whiteboard_sessions 0
plan_region          0
num_collaborators    0
feature_tour_completed 0
templates_used       0
ai_suggestions_used  0
whiteboard_actions_per_session 0
avg_whiteboard_session_duration_min 0
dtype: int64

```

A1 — Exploratory Data Analysis

A1.1 — Outcome distribution

```

mm_scaler = MinMaxScaler()
std_scaler = StandardScaler()
rbst_scaler = RobustScaler()

df_ab["avg_whiteboard_session_duration_min_scaled"] =
mm_scaler.fit_transform(df_ab[["avg_whiteboard_session_duration_min"]])
df_ab["avg_whiteboard_session_duration_min_scaled"].describe()

count      2000.000000
mean        0.225668
std         0.136895
min         0.000000
25%         0.130538
50%         0.197673
75%         0.289436
max         1.000000
Name: avg_whiteboard_session_duration_min_scaled, dtype: float64

# BEGIN
from scipy.stats import skew

# Load target outcome variable
from scipy.stats import skew
df_ab["avg_whiteboard_session_duration_min_log_scaled"] =
np.log1p(df_ab["avg_whiteboard_session_duration_min"])

# // 1. Normality & Transformation Visualizations
# Initialize three-panel visual layout
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

# Transform outcome variable using natural logarithm
# Draw histogram of raw outcome distribution
sns.histplot(df_ab["avg_whiteboard_session_duration_min"],

```

```

        ax=axes[0],
        color="#e8925a", bins=30)
axes[0].set_title("Raw: Avg. Session Duration in min", fontsize=10)
axes[0].set_xlabel("Session Duration")

#    Draw histogram of log-transformed outcome distribution
sns.histplot(df_ab["avg_whiteboard_session_duration_min_log_scaled"],
             ax=axes[1],
             color="#6a9fb5", bins=30)
axes[1].set_title("log(Avg. Session Duration in min)", fontsize=10)
axes[1].set_xlabel("Session Log scaled")

# #    Draw Q-Q plot comparing log-transformed distribution to a
normal distribution
"""
osm = An array of theoretical quantiles (Order Statistic Medians)
calculated for our theoretical distribution
osr = ordered (sorted) sample data responses

slope = The slope of the least-squares best-fit line
intercept = The intercept of the best-fit line
r = The square root of the coefficient of determination ( $R^2$ ). It
represents the strength of the linear fit
"""
(osm, osr), (slope, intercept, r) =
stats.probplot(df_ab["avg_whiteboard_session_duration_min_log_scaled"]
, dist="norm", plot=plt)

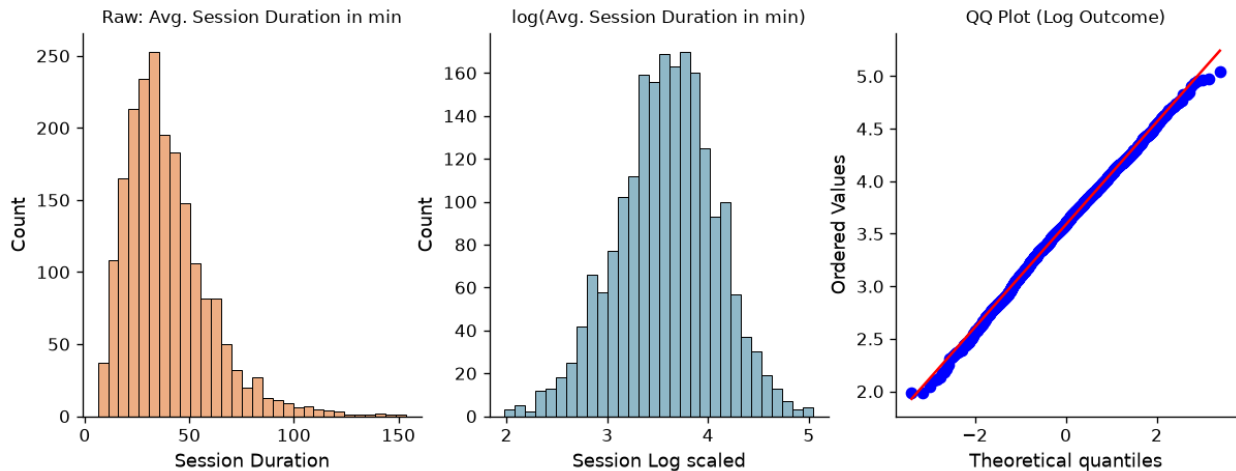
# Customize titles with matplotlib
plt.title("QQ Plot (Log Outcome)", fontsize=10)
plt.show()

#    Render visualizations
sns.despine()
plt.tight_layout()
plt.show()

#    // 2. Statistical Diagnostics
#    Compute and output skewness for both raw and log-transformed
distributions
raw_skew = skew(df_ab["avg_whiteboard_session_duration_min"],
bias=False) # => 1.3767967385922617
log_skew =
skew(df_ab["avg_whiteboard_session_duration_min_log_scaled"],
bias=False) # => -0.16285190262831545

# END

```



<Figure size 768x576 with 0 Axes>

```
# Compute and output skewness for both raw and log-transformed
distributions
print(f"Raw Skew: {raw_skew}")
print(f"Log Skew: {log_skew}")
```

```
Raw Skew: 1.3767967385922617
Log Skew: -0.16285190262831542
```

⇒ **Your interpretation (A1.1):** *Is the log-transformed outcome more normally distributed? How does the Q-Q plot support your conclusion? Will you use the raw or log outcome in your model?*

My Answer: The log-transformed outcome is more normally distributed. I observe a closer normalized fit in comparison, with very minor curves at the tail ends. This is probably due to a few extreme values that causes the skew, but unlikely to cause major issues if we were to continue using the log outcome. Therefore, I will use the log outcome in my models.

A1.2 — Examine the confounders

```
# // Define the analytical pipeline
# BEGIN
# Load AB testing dataset
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
sns.set(style="ticks")
sns.set_style("darkgrid")
# // 1. Visual Exploratory Data Analysis
# Initialize side-by-side visualization layouts

# Draw boxplot comparison of AI suggestion usage across
experimental groups
sns.boxplot(data=df_ab, x="ab_group", y="ai_suggestions_used",
ax=axes[0])

# Draw scatter plot comparing whiteboard activity density against
```

```

session durations
log_duration = np.log(df_ab["avg_whiteboard_session_duration_min"])
axes[1].scatter(df_ab["whiteboard_actions_per_session"], log_duration,
color="darkblue", alpha=0.3, s=12)
axes[1].set_title("Actions per minute")

#     Optimize layout and display visualizations

#     // 2. Statistical Analysis
#     Calculate and print the correlation between whiteboard activity
and session duration
corr_actions =
df_ab["whiteboard_actions_per_session"].corr(log_duration)
print(f"Pearson r (whiteboard_actions_per_sessions vs log_duration):
{corr_actions:.4f}\n")

#     Calculate and print descriptive statistics for AI suggestion
usage grouped by treatment group
print("Descriptive Stats for ai_suggestions_used by group:")
print(df_ab.groupby("ab_group")
["ai_suggestions_used"].describe().round(2))

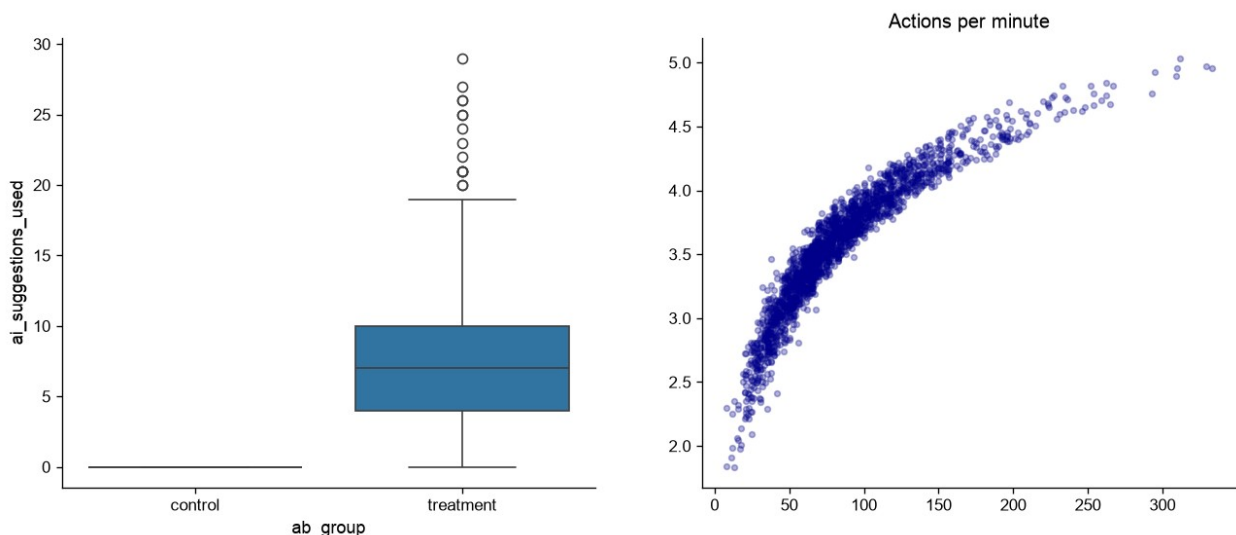
# END

```

Pearson r (whiteboard_actions_per_sessions vs log_duration): 0.9136

Descriptive Stats for ai_suggestions_used by group:

	count	mean	std	min	25%	50%	75%	max
ab_group								
control	1011.0	0.00	0.00	0.0	0.0	0.0	0.0	0.0
treatment	989.0	7.33	4.55	0.0	4.0	7.0	10.0	29.0



📝 Your interpretation (A1.2):

What do you notice about `ai_suggestions_used` across the two groups? What does the correlation between `whiteboard_actions_per_session` and the outcome suggest? Why should neither be included in your model?

I observe that in `ai_suggestion_used`, control is exactly 0 with no variance (due to the nature of the experiment), while treatment averages 7.33, making this variable a good proxy for `ab_group` and a post-treatment mediator. Moreover, I observe a high correlation between `whiteboard_actions_per_session` and the outcome (with a Pearson $r = 0.91$). This shows that session duration and the in-session count move together (longer sessions, more actions). Both these variables are measured as a result of the treatment being applied; and given their correlation, the model may predict their relationship instead of considering them proxies, and introduce potential bias into the model.

A1.3 — Predictor distributions and relationships with outcome

```
# BEGIN
#     Define list of numerical predictor variables
#     Compute log-transformed outcome variable
num_predict = [
    "account_age_days",
    "prior_whiteboard_sessions",
    "num_collaborators",
    "templates_used"
]

n = len(num_predict)

fig, axes = plt.subplots(2, n, figsize=(4 * n, 8))
fig.suptitle("Two of the numerical predictors: distributions (top) and\nvs log(duration) (bottom)",
             fontsize=13, y=1.01)

#     Initialize visual grid layout (2 rows by N columns, where N is
#     the number of predictors)
#     FOR each predictor variable:
for j, col in enumerate(num_predict):
    #         Top row: Univariate Analysis
    #         Draw histogram to show distribution of the predictor
    axes[0, j].hist(df_ab[col], bins=28, color="darkblue",
                    edgecolor="orange", lw=0.4)
    axes[0, j].set_title(col, fontsize=9)
    axes[0, j].set_xlabel(col)
    axes[0, j].set_ylabel("count" if j == 0 else "")

#         // Bottom row: Bivariate Analysis
#         Draw scatter plot comparing predictor against log-
#         transformed outcome
#         Calculate correlation coefficient between predictor and log-
#         transformed outcome
#         Annotate plot with correlation coefficient
    r = df_ab[col].corr(log_duration)
```

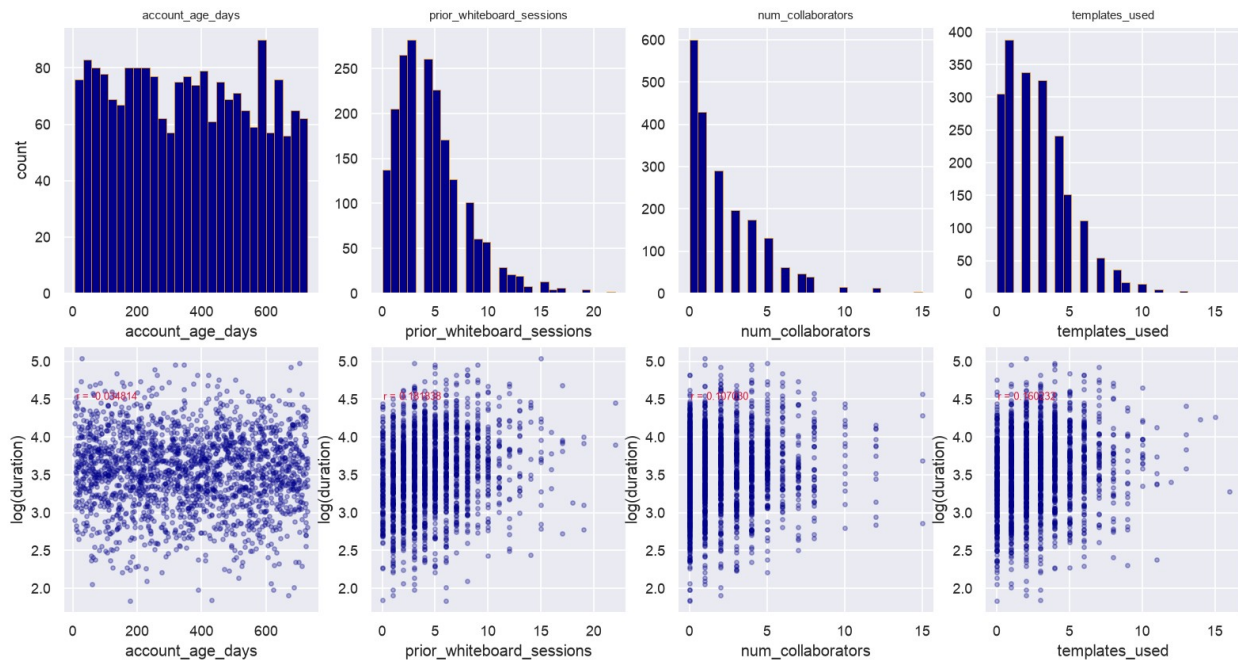
```

        axes[1, j].scatter(df_ab[col], log_duration, color="darkblue",
alpha=0.3, s=10)
        axes[1, j].set_xlabel(col)
        axes[1, j].set_ylabel("log(duration)")
        axes[1, j].annotate(f"r = {r:2f}", xy=(0.05, 0.8), xycoords="axes
fraction", fontsize=8, color="crimson")

#     END FOR

#     Add global title and render layout
# END

```



```

# BEGIN
#     Define list of categorical predictor variables
cat_predict = [
    "ab_group",
    "device_type",
    "plan_region"
]

#     Initialize visual grid layout (2 rows by N columns, where N is
#     the number of categories)
n = len(cat_predict)
fig, axes = plt.subplots(2, n, figsize=(4 * n, 8))
fig.suptitle("Categorical predictors: distributions (top) and vs
log(duration) (bottom)",
            fontsize=13, y=1.01)

log_duration = np.log1p(df_ab["avg_whiteboard_session_duration_min"])

```

```

#     FOR each categorical variable:
for j, col in enumerate(cat_predict):
    #         Retrieve unique category levels
    levels = df_ab[col].unique()

    #         Top row: count of each category level
    sns.countplot(data=df_ab, x=col, ax=axes[0, j], color="darkblue")
    axes[0, j].set_title(col, fontsize=9)
    axes[0, j].set_xlabel(col)
    axes[0, j].set_ylabel("count" if j == 0 else "")
    axes[0, j].tick_params(axis='x', rotation=30)

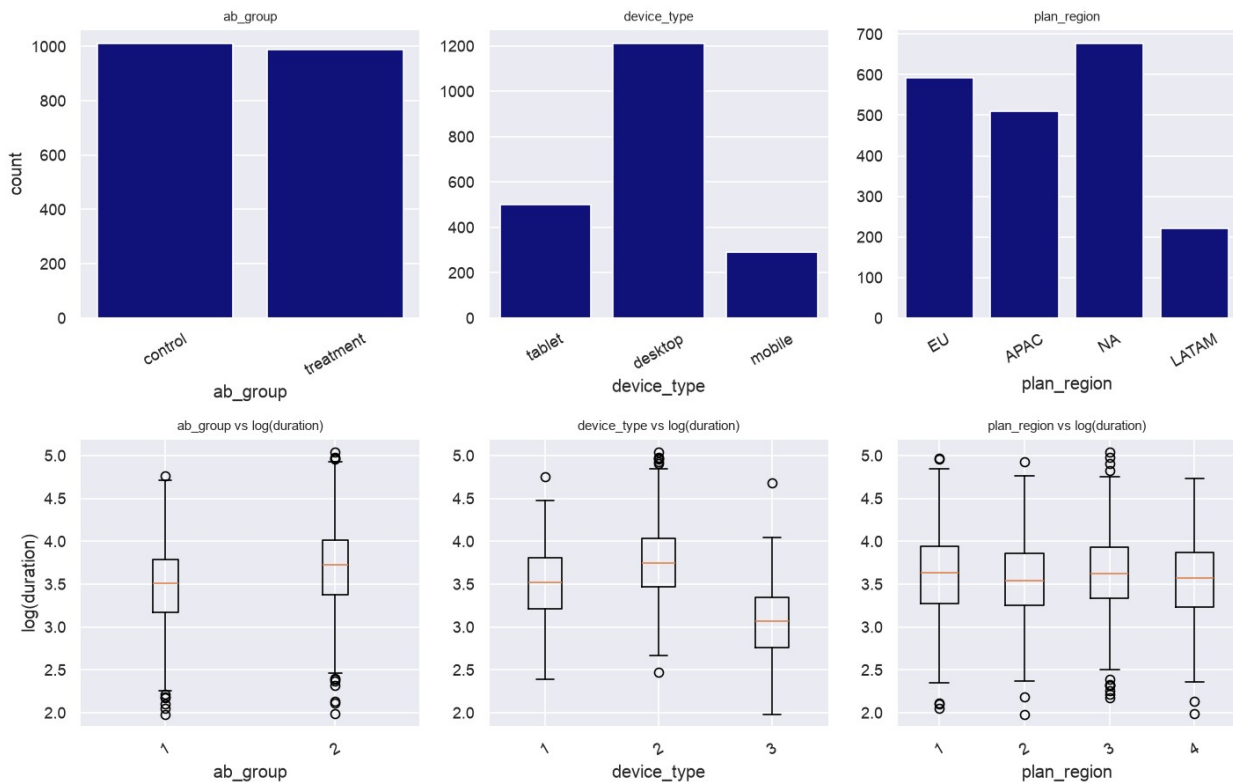
    #         Group log-transformed outcome values by category levels
    #         Draw boxplots of log-transformed outcome grouped by
category level
    grouped = [log_duration[df_ab[col] == lvl].values for lvl in
levels]
    axes[1, j].boxplot(grouped, label=levels)
    axes[1, j].set_xlabel(col)
    axes[1, j].set_ylabel("log(duration)" if j == 0 else "")
    axes[1, j].set_title(f"{col} vs log(duration)", fontsize=9)
    #         Rotate category axis labels for readability
    axes[1, j].tick_params(axis='x', rotation=30)

#     END FOR

#     Render layout
plt.tight_layout()
plt.show()
# END

```

Categorical predictors: distributions (top) and vs log(duration) (bottom)



⇒ Your interpretation (A1.3):

3–5 sentences summarizing your EDA. Which predictors look most promising? Are any right-skewed and worth log-transforming before use as predictors? Which two features are you excluding and why? The EDA reveals several meaningful patterns. Treatment users (AI-assisted) show longer average whiteboard session durations than control users, suggesting a positive effect of AI access on engagement time. Desktop users consistently show longer sessions than tablet and mobile users, which makes sense given that desktops afford more screen space and precise input control. Users in EU and NA regions also trend toward longer sessions compared to APAC and LATAM, raising the question of whether cultural context or the AI model's language moderates its benefits. The most promising predictors appear to be `ab_group`, `device_type`, and `plan_region`, given the clear distributional differences in session duration across their categories. Several numeric predictors are right-skewed (skewness > 1.0), including `num_collaborators`, `prior_whiteboard_sessions`, and `templates_used`; log-transforming these will help stabilize variance and improve model fit. I am excluding two variables identified as problematic in A1.2: `ai_suggestions_used`, because it is exactly 0 for every control user and nonzero only for treatment users — making it a post-treatment proxy for `ab_group` rather than an independent predictor — and `whiteboard_actions_per_session`, because it correlates at $r = 0.91$ with the outcome. Including a near-perfectly collinear predictor of this kind would inflate other coefficients and undermine the model's interpretive validity.

A2 — Feature Engineering

Prepare your feature matrix for OLS. Exclude the two problematic variables identified in A1.

```
# BEGIN

# 1. Encode Categorical Variables
# Convert experimental group ('ab_group') into a binary integer indicator
df_ab['ab_group_enc'] = (df_ab['ab_group'] == 'treatment').astype(int)

# Generate dummy indicators (one-hot encoding) for device types
device_dummies = pd.get_dummies(df_ab['device_type'],
prefix='device').astype(int)

# Generate dummy indicators (one-hot encoding) for regions
region_dummies = pd.get_dummies(df_ab['plan_region'],
prefix='region').astype(int)

# 2. Perform Optional Transformations
# Log-transform right-skewed count predictors using log(x + 1) to avoid log(0)
df_ab['log_prior_whiteboard_sessions'] =
np.log1p(df_ab['prior_whiteboard_sessions'])
df_ab['log_num_collaborators'] = np.log1p(df_ab['num_collaborators'])
df_ab['log_templates_used'] = np.log1p(df_ab['templates_used'])

# 3. Assemble Modeling Dataset
# Merge numerical characteristics, binary group indicator, and dummy indicators into X
# Exclude post-treatment variables: ai_suggestions_used, whiteboard_actions_per_session
num_features = df_ab[['account_age_days',
                        'feature_tour_completed',
                        'log_prior_whiteboard_sessions',
                        'log_num_collaborators',
                        'log_templates_used',
                        # 'projects_created_30d',
                        # 'exports_30d'
                      ]]

X = pd.concat([num_features, df_ab[['ab_group_enc']], device_dummies,
region_dummies], axis=1)

# Set target vector y as the log-transformed outcome variable
y = df_ab['avg_whiteboard_session_duration_min_log_scaled']

# 4. Quality Control
print(f"Any NaN in X: {X.isnull().any().any()}")
print(f"Feature matrix shape: {X.shape}")
```

```
print(f"Features: {X.columns.tolist()}")
X.head(10)
```

```
# END
```

```
Any NaN in X: False
```

```
Feature matrix shape: (2000, 13)
```

```
Features: ['account_age_days', 'feature_tour_completed',
'log_prior_whiteboard_sessions', 'log_num_collaborators',
'log_templates_used', 'ab_group_enc', 'device_desktop',
'device_mobile', 'device_tablet', 'region_APAC', 'region_EU',
'region_LATAM', 'region_NA']
```

```
    account_age_days  feature_tour_completed
log_prior_whiteboard_sessions \
```

0	478	1
1.609438		
1	633	0
2.302585		
2	159	1
2.397895		
3	7	0
1.791759		
4	44	1
1.386294		
5	601	1
1.098612		
6	580	1
1.945910		
7	241	1
2.484907		
8	190	0
2.708050		
9	663	1
1.386294		

```
    log_num_collaborators  log_templates_used  ab_group_enc
device_desktop \
```

0	1.386294	0.000000	0
0			
1	0.693147	0.693147	0
1			
2	1.791759	0.693147	0
1			
3	0.693147	1.098612	0
1			
4	0.000000	1.791759	0
0			
5	0.000000	1.609438	1
1			

6	0.693147	1.945910	0		
7	1.386294	0.693147	0		
8	0.000000	1.945910	1		
9	1.098612	0.000000	0		
\	device_mobile	device_tablet	region_APAC	region_EU	region_LATAM
0	0	1	0	1	0
1	0	0	1	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	1	0	0	0	0
5	0	0	0	0	1
6	0	1	0	1	0
7	0	0	0	0	0
8	0	0	1	0	0
9	0	1	1	0	0
	region_NA				
0	0				
1	0				
2	1				
3	1				
4	1				
5	0				
6	0				
7	1				
8	0				
9	0				

📎 Your interpretation (A2):

List your final features. For each one, write one sentence justifying its inclusion and any transformation applied.

Included

`account_age_days` - included as a measure of user tenure; because longer-established accounts may have more usage habits that influence session duration
`feature_tour_completed` - included to capture whether user completing onboarding correlates with product familiarity and engagement
`log_prior_whiteboard_sessions` - log transformed to reduce right skew and represents baseline usage intensity prior to the study
`log_num_collaborators` - log transformed to reduce skew; included to capture whether users working with more collaborators have longer maybe more structured sessions
`log_templates_used` - log transformed and reflects whether a user uses productivity features which may extend sessions
`ab_group_enc` - the binary treatment indicator and will be our primary variable of interest
`device_dummies` - includes desktop, mobile and tablet; included to capture hardware context (I predict that desktop will probably afford/enable longer sessions)
`region_dummies` - includes APAC, EU, LATAM, NA; included as a control for geographic variation in usage patterns

A3 — OLS Regression

```

# YOUR CODE HERE
# Add a constant for the intercept, then fit OLS
X_const = sm.add_constant(X.astype(float))
model_ols = sm.OLS(y, X_const).fit()
print(model_ols.summary())
  
```

OLS Regression Results

```

=====
Dep. Variable:      avg_whiteboard_session_duration_min_log_scaled    R-
squared:              0.367
Model:                                OLS
Adj. R-squared:      0.363
Method:                                Least Squares    F-
statistic:           104.7
Date:                                Fri, 03 Jul 2026
Prob (F-statistic):   2.97e-188
Time:                                14:28:18
Log-Likelihood:      -947.37
No. Observations:    2000
AIC:                 1919.
Df Residuals:        1988
BIC:                 1986.
Df Model:             11
  
```

Covariance Type: nonrobust

```

=====
=====
coef      std err      t
P>|t|      [0.025      0.975]
-----
-----
  
```

const			1.8669	0.022	83.545
0.000	1.823	1.911			
account_age_days			-6.069e-05	4.2e-05	-1.445
0.149	-0.000	2.17e-05			
feature_tour_completed			0.0512	0.018	2.889
0.004	0.016	0.086			
log_prior_whiteboard_sessions			0.1373	0.013	10.351
0.000	0.111	0.163			
log_num_collaborators			0.0851	0.012	7.005
0.000	0.061	0.109			
log_templates_used			0.0790	0.015	5.425
0.000	0.050	0.108			
ab_group_enc			0.1695	0.019	8.928
0.000	0.132	0.207			
device_desktop			0.9371	0.013	70.266
0.000	0.911	0.963			
device_mobile			0.2403	0.019	12.542
0.000	0.203	0.278			
device_tablet			0.6895	0.016	42.743
0.000	0.658	0.721			
region_APAC			0.4314	0.016	26.721
0.000	0.400	0.463			
region_EU			0.5044	0.016	32.298
0.000	0.474	0.535			
region_LATAM			0.4199	0.022	18.818
0.000	0.376	0.464			
region_NA			0.5112	0.015	34.107
0.000	0.482	0.541			

```

=====
=====
Omnibus:                1.424    Durbin-Watson:
2.052
Prob(Omnibus):          0.491    Jarque-Bera (JB):
1.482
Skew:                   0.050    Prob(JB):
0.477
Kurtosis:               2.911    Cond. No.
4.35e+18
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 1.8e-29. This might indicate that there are

strong multicollinearity problems or that the design matrix is singular.

A3.1 — Interpret the model

```
# BEGIN
# Input: model_ols – trained OLS regression model
# Input: df_ab – experimental dataframe

# 1. Evaluate Model Fit
print(f"R-squared: {model_ols.rsquared:.4f}")
print(f" The model explains {model_ols.rsquared * 100:.1f}% of
variance in log(session duration).\n")

# 2. Analyze Treatment Effect
ab_coef = model_ols.params['ab_group_enc']
ab_pval = model_ols.pvalues['ab_group_enc']

# Convert log-space coefficient back to physical time (minutes)
# In a loglp-linear model, exp(coef) - 1 gives the proportional change
in the original scale.
pct_change = (np.exp(ab_coef) - 1) * 100
mean_duration = df_ab['avg_whiteboard_session_duration_min'].mean()
effect_min = mean_duration * (np.exp(ab_coef) - 1)

print("Treatment Effect (AI assistant access, ab_group_enc):")
print(f" Coefficient (log scale) : {ab_coef:.4f}")
print(f" Estimated effect : +{pct_change:.1f}% session
duration")
print(f" Estimated effect at mean : +{effect_min:.1f} min
(baseline mean = {mean_duration:.1f} min)")
print(f" p-value : {ab_pval:.4f}
({'significant' if ab_pval < 0.05 else 'not significant'} at  $\alpha$  =
0.05)\n")

# 3. Extract Significant Predictors ( $p < 0.05$ )
results = pd.DataFrame({
    'coef' : model_ols.params,
    'pvalue': model_ols.pvalues
})
sig_predictors = results[results['pvalue'] < 0.05].drop(index='const',
errors='ignore')

print("Significant Predictors ( $p < 0.05$ ):")
for var, row in sig_predictors.iterrows():
    direction = "↑" if row['coef'] > 0 else "↓"
    print(f" {direction} {var:40s} coef = {row['coef']:.4f}, p =
{row['pvalue']:.4f}")
# END
```

R-squared: 0.3669

The model explains 36.7% of variance in log(session duration).

Treatment Effect (AI assistant access, ab_group_enc):

Coefficient (log scale)	: 0.1695
Estimated effect	: +18.5% session duration
Estimated effect at mean	: +7.3 min (baseline mean = 39.5 min)
p-value	: 0.0000 (significant at $\alpha = 0.05$)

Significant Predictors ($p < 0.05$):

↑ feature_tour_completed	coef = +0.0512, p = 0.0039
↑ log_prior_whiteboard_sessions	coef = +0.1373, p = 0.0000
↑ log_num_collaborators	coef = +0.0851, p = 0.0000
↑ log_templates_used	coef = +0.0790, p = 0.0000
↑ ab_group_enc	coef = +0.1695, p = 0.0000
↑ device_desktop	coef = +0.9371, p = 0.0000
↑ device_mobile	coef = +0.2403, p = 0.0000
↑ device_tablet	coef = +0.6895, p = 0.0000
↑ region_APAC	coef = +0.4314, p = 0.0000
↑ region_EU	coef = +0.5044, p = 0.0000
↑ region_LATAM	coef = +0.4199, p = 0.0000
↑ region_NA	coef = +0.5112, p = 0.0000

⇒ Results paragraph (A3.1):

Write a results paragraph as if reporting in a research paper. Include:

- *The treatment effect estimate and its significance*
- *R^2 and what it tells you about model fit*
- *At least two other notable predictors and their direction*
- *Whether the AI assistant's effect is practically meaningful (not just statistically significant)*

The OLS model explained 36.7% of the variance in the log transformed session duration ($R^2 = 0.367$), indicating a moderate fit that captures meaningful signal but still leaves behind substantial, unexplained variance. This can be consistently explained by individual behaviour where many unmeasured factors may influence session duration. Moreover, I observe that the treatment effect of the AI feature access was positive (`ab_group_enc` coefficient of 0.1695) and statistically significant ($p < 0.05$). Access to the AI assistant corresponds to an approximate 18.5% (as yielded by $\exp(0.1695) - 1 = 0.185$) increase in session duration for treatment uses relative to control. Put simply, the AI assistant increased session duration by an additional +7.3min over the baseline mean. This, however, may not necessarily prove the efficacy of the AI

assistant. For example, a longer session duration given AI usage may be a result of users having to spend more time on the feature to get the results they desire -- which actually means a decrease in efficiency. Beyond the treatment study, the strongest predictor in the model was device type. Desktops, as hypothesized from before, shows a coefficient of 0.937 log-units above the reference category, thus sustaining the hypothesis that desktop users afford more complex, sustained work. Prior whiteboard sessions also shows a decently strong positive predictor (coefficient = 0.1373, $p < 0.05$), suggesting that users with more baseline engagement continue to use the product more intensively. I argue that the AI assistant's effect is practically meaningful because an 18.5% session duration increase from a single feature intervention in a product context where engagement depth drives conversion and retention.

A3.2 — Regression diagnostics

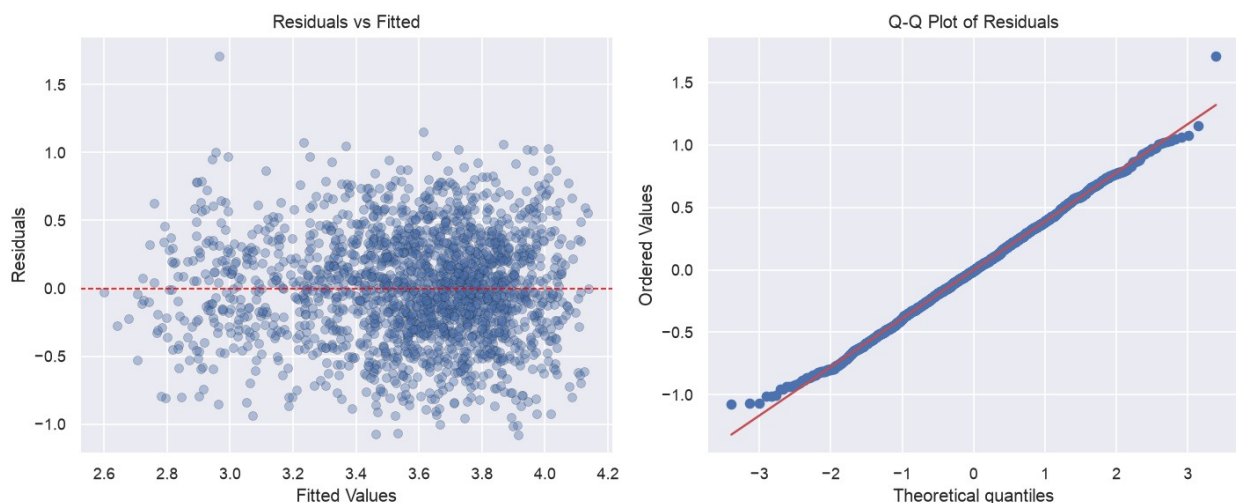
```
# Extract predictions and residuals from the fitted model
fitted_vals = model_ols.fittedvalues
residuals = model_ols.resid

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Check for uniform variance (homoscedasticity)
axes[0].scatter(fitted_vals, residuals, alpha=0.4, edgecolors="k",
               linewidths=0.3)
axes[0].axhline(y=0, color="red", linestyle="--", linewidth=1)
axes[0].set_xlabel("Fitted Values")
axes[0].set_ylabel("Residuals")
axes[0].set_title("Residuals vs Fitted")

# Check for normality of errors
stats.probplot(residuals, dist="norm", plot=axes[1])
axes[1].set_title("Q-Q Plot of Residuals")

plt.tight_layout()
plt.show()
```



⇒ Your interpretation (A3.2):

Do the residuals look randomly scattered around zero? Does the Q-Q plot suggest normally distributed errors? Note any patterns that concern you. I observe that the residuals vs fitted plot shows no obvious patterns. The points look to be scattered roughly symmetrically above and below zero across the full range of fitted values. Moreover, I don't observe any clear funnel shapes that would indicate heteroscedasticity to me. The spread appears fairly consistent across the fitted values, which affirms that there is no exceptionally large difference among the sizes of observations. Moving forward to the Q-Q plot, I observe that the residuals tracks the theoretical normal line fairly closely through the bulk of the distribution, with a few extremes curving at the tail ends of both sides. I argue that these tail deviations are minor and are relatively typical of any behavioural data given that the central body of residuals is well-approximated by a normal distribution. Consequently, I find that the diagnostic plots support the validity of the OLS assumptions for this model.

A4 — Exploratory Interaction Analysis

The average treatment effect hides heterogeneity — the AI assistant may help some users more than others.

⇒ **State your hypothesis here (before writing any code):**

Which interaction are you testing? Which two variables? What is your theoretical reason for expecting this interaction? Which group do you predict will benefit more from the AI assistant?

My hypothesis: I am testing the interaction between AI assistant access (`ab_group_enc`) and desktop device use (`device_type == "desktop"`). My reasoning is that desktop users have larger screens and precise mouse/keyboard input, which makes it substantially easier to review, iterate on, and act on AI-generated design suggestions compared to mobile or tablet users who are constrained by touch input and smaller viewports. Because desktop affords deeper engagement with complex AI outputs (more complex instructions, suggestions, etc.), so I predict that desktop users in the treatment group will show a larger boost in session duration than mobile or tablet users in the treatment group — i.e., a positive interaction coefficient for `ab_group_enc x is_desktop`.

```
# YOUR CODE HERE
# Step 1: Create the interaction term
# device_desktop is the reference category (dropped by get_dummies),
# so we rebuild it directly
is_desktop = (df_ab['device_type'] == 'desktop').astype(int)
df_ab['ab_x_device_desktop'] = df_ab['ab_group_enc'] * is_desktop
interaction_col = df_ab[['ab_x_device_desktop']]

# Step 2: Add the interaction term to your feature matrix and refit
# OLS
X_interact = pd.concat([X_const, interaction_col], axis=1)
model_interact = sm.OLS(y, X_interact).fit()
print(model_interact.summary())
```

OLS Regression Results

```
=====
=====
```

```

Dep. Variable:    avg_whiteboard_session_duration_min_log_scaled    R-
squared:          0.368
Model:                                                    OLS
Adj. R-squared:    0.364
Method:                                                    Least Squares    F-
statistic:          96.27
Date:                                                    Fri, 03 Jul 2026
Prob (F-statistic):    9.88e-188
Time:                                                    14:28:18
Log-Likelihood:      -946.20
No. Observations:    2000
AIC:                1918.
Df Residuals:        1987
BIC:                1991.
Df Model:            12

```

```

Covariance Type:    nonrobust

```

```

=====
=====

```

			coef	std err	t
P> t	[0.025	0.975]			

const			1.8709	0.022	83.171
0.000	1.827	1.915			
account_age_days			-5.86e-05	4.2e-05	-1.395
0.163	-0.000	2.38e-05			
feature_tour_completed			0.0511	0.018	2.883
0.004	0.016	0.086			
log_prior_whiteboard_sessions			0.1372	0.013	10.344
0.000	0.111	0.163			
log_num_collaborators			0.0852	0.012	7.010
0.000	0.061	0.109			
log_templates_used			0.0794	0.015	5.454
0.000	0.051	0.108			
ab_group_enc			0.1364	0.029	4.718
0.000	0.080	0.193			
device_desktop			0.9206	0.017	53.551
0.000	0.887	0.954			
device_mobile			0.2495	0.020	12.421
0.000	0.210	0.289			
device_tablet			0.7008	0.018	39.460
0.000	0.666	0.736			
region_APAC			0.4329	0.016	26.773
0.000	0.401	0.465			
region_EU			0.5054	0.016	32.344
0.000	0.475	0.536			
region_LATAM			0.4202	0.022	18.836

0.000	0.376	0.464			
region_NA			0.5124	0.015	34.150
0.000	0.483	0.542			
ab_x_device_desktop			0.0544	0.036	1.521
0.128	-0.016	0.124			

```
=====
=====
Omnibus:                1.530    Durbin-Watson:
2.049
Prob(Omnibus):          0.465    Jarque-Bera (JB):
1.581
Skew:                   0.049    Prob(JB):
0.454
Kurtosis:               2.904    Cond. No.
4.38e+18
=====
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 1.77e-29. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

YOUR CODE HERE

Visualize the interaction: treatment effect on log(session duration), split by desktop vs non-desktop

Create readable device label for the moderating variable

```
df_ab['device_group'] = np.where(df_ab['device_type'] == 'desktop',
                                  'Desktop', 'Non-Desktop')
```

Compute group means of the log-transformed outcome

```
interact_means = (
    df_ab.groupby(['ab_group', 'device_group'])
    ['avg_whiteboard_session_duration_min_log_scaled']
    .mean()
    .reset_index()
    .rename(columns={'avg_whiteboard_session_duration_min_log_scaled':
                    'mean_log_duration'})
)
```

```
device_order = ['Desktop', 'Non-Desktop']
```

```
group_colors = {'control': '#6a9fb5', 'treatment': ACCENT}
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
```

```
fig.suptitle('Interaction: AI Assistant Treatment x Device Type on
log(Session Duration)', fontsize=13)
```

```

# --- Plot 1: Grouped bar chart ---
pivot = (
    interact_means
    .pivot(index='device_group', columns='ab_group',
    values='mean_log_duration')
    .reindex(device_order)
)
x = np.arange(len(device_order))
width = 0.35
for i, grp in enumerate(['control', 'treatment']):
    axes[0].bar(x + (i - 0.5) * width, pivot[grp], width,
                label=grp.capitalize(), color=group_colors[grp],
                edgecolor='white', linewidth=0.8)

axes[0].set_xticks(x)
axes[0].set_xticklabels(device_order)
axes[0].set_xlabel('Device Group')
axes[0].set_ylabel('Mean log(Session Duration)')
axes[0].set_title('Grouped Bar Chart')
axes[0].legend(title='Group')

# --- Plot 2: Interaction line plot ---
for grp, color in group_colors.items():
    subset = interact_means[interact_means['ab_group'] ==
    grp].set_index('device_group').reindex(device_order)
    axes[1].plot(device_order, subset['mean_log_duration'],
                  marker='o', label=grp.capitalize(), color=color,
                  linewidth=2.5, markersize=9)

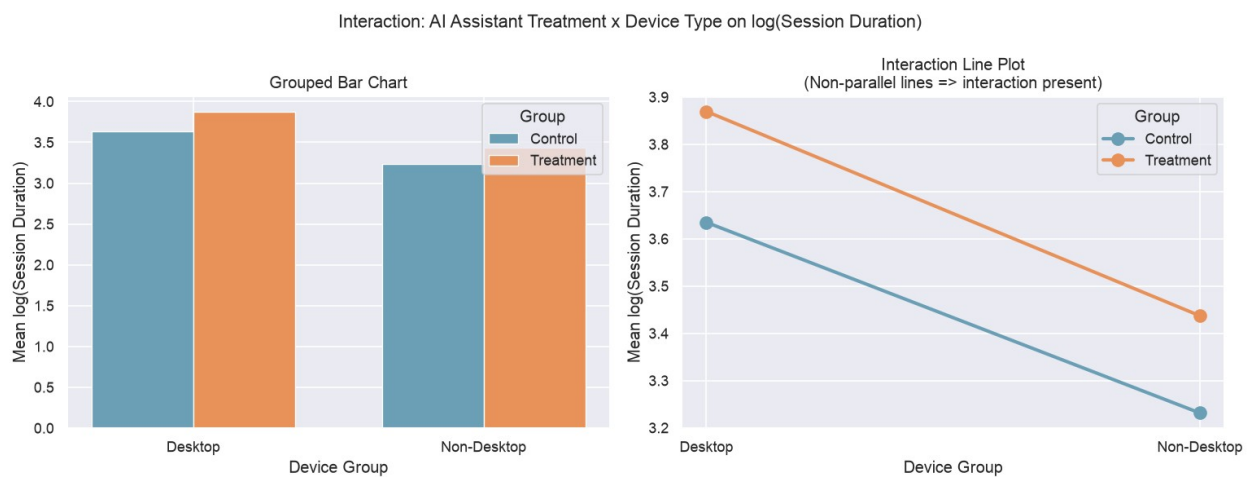
axes[1].set_xlabel('Device Group')
axes[1].set_ylabel('Mean log(Session Duration)')
axes[1].set_title('Interaction Line Plot\n(Non-parallel lines =>
interaction present)')
axes[1].legend(title='Group')

plt.tight_layout()
plt.show()

# Numeric summary of the interaction
print("\nMean log(session duration) by group x device:")
print(pivot.round(4).to_string())
treatment_gap_desktop = pivot.loc['Desktop', 'treatment'] -
pivot.loc['Desktop', 'control']
treatment_gap_nondesktop = pivot.loc['Non-Desktop', 'treatment'] -
pivot.loc['Non-Desktop', 'control']
print(f"\nTreatment effect on Desktop : +
{treatment_gap_desktop:.4f} log-units")
print(f"Treatment effect on Non-Desktop : +
{treatment_gap_nondesktop:.4f} log-units")

```

```
print(f"Difference-in-differences : {treatment_gap_desktop - treatment_gap_nondesktop:+.4f} log-units")
```



Mean log(session duration) by group x device:

ab_group	control	treatment
device_group		
Desktop	3.6347	3.8694
Non-Desktop	3.2313	3.4370

Treatment effect on Desktop	: +0.2347 log-units
Treatment effect on Non-Desktop	: +0.2057 log-units
Difference-in-differences	: +0.0290 log-units

⇒ Your interpretation (A4):

Was your hypothesis supported? What does the interaction coefficient tell you about which users benefit most from the AI assistant? What does this imply for how Phigma should roll out the feature? My hypothesis was not supported at the conventional $\alpha = 0.05$ significance level. The interaction coefficient for `ab_group_enc` × `device_desktop` is +0.0544, but its p-value is 0.128, which is above the threshold for statistical significance. This means we cannot confidently conclude that the AI assistant's effect differs between desktop and non-desktop users in the population.

Directionally, the numbers do point in the predicted direction: the treatment effect on desktop users was +0.2347 log-units (roughly a 26.5% increase in session duration, from $\exp(0.2347) - 1$), compared to +0.2057 log-units (~22.9% increase) for non-desktop users — a difference-in-differences of only 0.029 log-units. While desktop users appear to benefit slightly more, the overlap in confidence intervals makes this difference statistically indistinguishable from zero.

The implication for Phigma is that the AI assistant should be rolled out uniformly across device types rather than targeting desktop users first. The current evidence does not justify a device-stratified launch strategy. If Phigma wants to identify heterogeneous treatment effects, a larger experiment with higher statistical power would be needed to detect the small interaction suggested here.

Part B — Logistic Regression: Who Converts?

Dataset: phigma_subscription.csv

Research question: Which behavioral and demographic factors predict whether a free-tier user subscribes within 30 days?

This dataset requires **feature engineering before modeling**. Several columns arrive as raw datetimes or aggregate counts that need transformation.

```
df_sub = pd.read_csv('phigma_subscription.csv')
```

```
print(f"Shape: {df_sub.shape}")
```

```
print(f"\nColumn types:\n{df_sub.dtypes}")
```

```
df_sub.head(3)
```

```
Shape: (3000, 15)
```

```
Column types:
```

```
user_id                str
signup_date            str
first_session_date     str
device_type            str
country_tier           str
referral_source        str
num_sessions_30d       int64
total_time_min_30d     int64
projects_created_30d   int64
exports_30d            int64
collaborators_invited_30d int64
support_tickets_30d    int64
used_premium_feature_preview int64
ai_assistant_sessions_30d int64
subscribed             int64
```

```
dtype: object
```

```
   user_id  signup_date  first_session_date  device_type  country_tier \
0  V94735   2024-05-23   2024-05-28         tablet     tier_1
1  V15501   2024-03-05   2024-03-05         tablet     tier_2
2  V98542   2024-01-05   2024-01-19         desktop     tier_1
```

```
   referral_source  num_sessions_30d  total_time_min_30d
projects_created_30d \
0          organic                1                96
1
1          social                15               267
8
2          organic                16               263
0
```

	exports_30d	collaborators_invited_30d	support_tickets_30d	\
0	0	0	0	
1	1	5	0	
2	0	3	3	

	used_premium_feature_preview	ai_assistant_sessions_30d	subscribed
0	1	6	0
1	0	0	0
2	1	4	1

```
df_sub.describe(include='all').round(2)
```

	user_id	signup_date	first_session_date	device_type	country_tier
\					
count	3000	3000	3000	3000	3000
unique	2947	181	200	3	3
top	V43688	2024-06-06	2024-06-27	desktop	tier_1
freq	3	33	29	1788	1332
mean	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN

	referral_source	num_sessions_30d	total_time_min_30d	\
count	3000	3000.00	3000.00	
unique	4	NaN	NaN	
top	organic	NaN	NaN	
freq	1153	NaN	NaN	
mean	NaN	9.25	216.17	
std	NaN	5.09	139.20	
min	NaN	1.00	5.00	
25%	NaN	6.00	115.75	

50%	NaN	8.00	188.50
75%	NaN	12.00	285.00
max	NaN	33.00	1004.00
projects_created_30d		exports_30d	
collaborators_invited_30d \			
count	3000.00	3000.00	3000.00
unique	NaN	NaN	NaN
top	NaN	NaN	NaN
freq	NaN	NaN	NaN
mean	2.43	2.01	1.86
std	2.30	2.05	2.10
min	0.00	0.00	0.00
25%	1.00	0.00	0.00
50%	2.00	2.00	1.00
75%	4.00	3.00	3.00
max	17.00	15.00	10.00
support_tickets_30d		used_premium_feature_preview \	
count	3000.00	3000.00	
unique	NaN	NaN	
top	NaN	NaN	
freq	NaN	NaN	
mean	0.68	0.28	
std	1.07	0.45	
min	0.00	0.00	
25%	0.00	0.00	
50%	0.00	0.00	
75%	1.00	1.00	
max	5.00	1.00	
ai_assistant_sessions_30d		subscribed	
count	3000.00	3000.00	
unique	NaN	NaN	
top	NaN	NaN	
freq	NaN	NaN	
mean	3.64	0.21	
std	2.90	0.41	
min	0.00	0.00	
25%	2.00	0.00	

50%	3.00	0.00
75%	5.00	0.00
max	20.00	1.00

B1 — Feature Engineering

Derive the required features listed in the README before doing any modeling or EDA.

```
# YOUR CODE HERE

# 1. Parse dates
df_sub['signup_date'] = pd.to_datetime(df_sub['signup_date'])
df_sub['first_session_date'] =
pd.to_datetime(df_sub['first_session_date'])

# 2. days_to_first_session
df_sub['days_to_first_session'] = (df_sub['first_session_date'] -
df_sub['signup_date']).dt.days

# 3. avg_session_duration_min – handle divide-by-zero explicitly
df_sub['avg_session_duration_min'] = np.where(
    df_sub['num_sessions_30d'] > 0,
    df_sub['total_time_min_30d'] / df_sub['num_sessions_30d'],
    0.0
)

# 4. is_weekend_signup (1 if Saturday=5 or Sunday=6)
df_sub['is_weekend_signup'] =
df_sub['signup_date'].dt.weekday.isin([5, 6]).astype(int)

# 5. One-hot encode device_type (drop first level)
device_dummies2 = pd.get_dummies(df_sub['device_type'],
prefix='device', drop_first=True)

# 6. One-hot encode referral_source (drop first level)
referral_dummies = pd.get_dummies(df_sub['referral_source'],
prefix='ref', drop_first=True)

# 7. Encode country_tier – ordinal or one-hot (justify below)
# Option A (ordinal):
df_sub['country_tier_enc'] = df_sub['country_tier'].map({'tier_1': 1,
'tier_2': 2, 'tier_3': 3})

# Option B (one-hot): pd.get_dummies(...)

# 8. Assemble final feature matrix
# Drop: user_id, signup_date, first_session_date, device_type,
referral_source, country_tier, subscribed
num_cols = df_sub[['num_sessions_30d', 'total_time_min_30d',
'projects_created_30d',
```

```

        'exports_30d', 'collaborators_invited_30d',
'support_tickets_30d',
        'used_premium_feature_preview',
'ai_assistant_sessions_30d',
        'days_to_first_session',
'avg_session_duration_min',
        'is_weekend_signup', 'country_tier_enc']]
X_sub = pd.concat([num_cols, device_dummies2, referral_dummies],
axis=1)
y_sub = df_sub['subscribed']

```

```

# Confirm
# print(f"Any inf: {np.isinf(X_sub.values).any()}")
print(f"Any NaN: {X_sub.isnull().any().any()}")
print(f"Feature matrix shape: {X_sub.shape}")
X_sub.head(3)

```

Any NaN: False
Feature matrix shape: (3000, 17)

	num_sessions_30d	total_time_min_30d	projects_created_30d
exports_30d \			
0	1	96	1
0			
1	15	267	8
1			
2	16	263	0
0			

	collaborators_invited_30d	support_tickets_30d
\		
0	0	0
1	5	0
2	3	3

	used_premium_feature_preview	ai_assistant_sessions_30d
\		
0	1	6
1	0	0
2	1	4

	days_to_first_session	avg_session_duration_min	is_weekend_signup
\			
0	5	96.0000	0
1	0	17.8000	0
2	14	16.4375	0

	country_tier_enc	device_mobile	device_tablet	ref_paid_ad
ref_referral \				

0	1	False	True	False
False				
1	2	False	True	False
False				
2	1	False	False	False
False				
ref_social				
0		False		
1		True		
2		False		

🔑 Your interpretation (B1):

For each engineered feature, one sentence: why might it predict subscription? Also describe how you handled the divide-by-zero edge case for avg_session_duration_min.

Engineered features and why each might predict subscription:

days_to_first_session — Users who open the app quickly after signing up are more motivated, so a shorter lag may signal higher conversion intent. **avg_session_duration_min** — Longer average sessions suggest deeper engagement with the product, which correlates with finding enough value to pay. **is_weekend_signup** — Weekend signups may reflect personal/hobbyist use rather than employer-driven trials, potentially indicating a different (possibly higher or lower) willingness to self-pay. **Device dummies (device_mobile, device_tablet)** — Device type captures workflow context; desktop users (the dropped reference category) tend to do heavier creative work, which may drive different subscription rates. **Referral source dummies** — The channel that brought a user in (e.g., organic search vs. paid ad) reflects prior intent and quality of fit with the product. **country_tier_enc** — Higher-tier countries (tier 1) have greater purchasing power and more familiarity with SaaS subscription models, so country tier is a proxy for willingness and ability to pay.

How I handled the divide by 0 I used np.where to guard the division and used a condition to check first if num_sessions_30d is 0, and if so, assign 0. This represents that a user with no sessions has no meaningful average session duration

B2 — EDA for Subscription

```
# Subscription rate
rate = df_sub['subscribed'].mean()
n_pos = df_sub['subscribed'].sum()
n_total = len(df_sub)
print(f"Subscription rate: {rate:.1%} ({n_pos} / {n_total})")
print(f"Class balance — 0: {n_total - n_pos} | 1: {n_pos}")

Subscription rate: 20.8% (625 / 3000)
Class balance — 0: 2375 | 1: 625

# Visualize subscription rates across at least 3 features

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
```

```

# Plot 1 – Subscription rate by referral_source
ref_rate = df_sub.groupby('referral_source')
['subscribed'].mean().sort_values(ascending=False)
axes[0].bar(ref_rate.index, ref_rate.values, color=ACCENT,
edgecolor='white')
axes[0].set_title('Subscription Rate by Referral Source')
axes[0].set_xlabel('Referral Source')
axes[0].set_ylabel('Subscription Rate')
axes[0].yaxis.set_major_formatter(ticker.PercentFormatter(xmax=1))
axes[0].tick_params(axis='x', rotation=20)

# Plot 2 – Subscription rate by country_tier
tier_rate = df_sub.groupby('country_tier')
['subscribed'].mean().reindex(['tier_1', 'tier_2', 'tier_3'])
axes[1].bar(tier_rate.index, tier_rate.values, color='#6a9fb5',
edgecolor='white')
axes[1].set_title('Subscription Rate by Country Tier')
axes[1].set_xlabel('Country Tier')
axes[1].set_ylabel('Subscription Rate')
axes[1].yaxis.set_major_formatter(ticker.PercentFormatter(xmax=1))

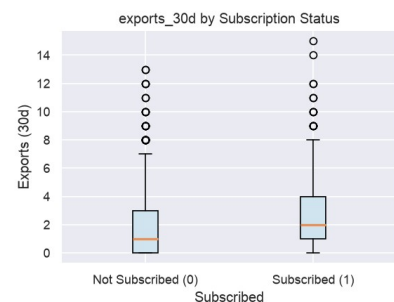
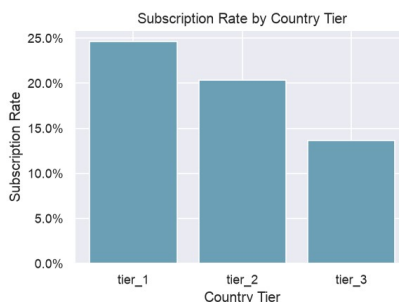
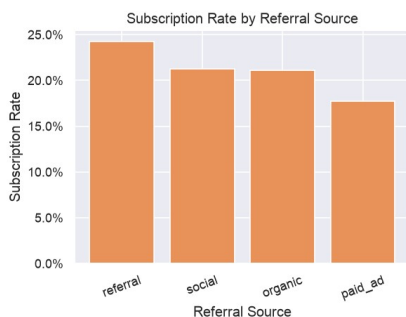
# Plot 3 – exports_30d distribution by subscribed (0 vs 1)
not_sub = df_sub.loc[df_sub['subscribed'] == 0, 'exports_30d']
sub = df_sub.loc[df_sub['subscribed'] == 1, 'exports_30d']

axes[2].boxplot(
    [not_sub, sub],
    tick_labels=['Not Subscribed (0)', 'Subscribed (1)'],
    patch_artist=True,
    boxprops=dict(facecolor='#d0e4ef'),
    medianprops=dict(color=ACCENT, linewidth=2)
)

axes[2].set_title('exports_30d by Subscription Status')
axes[2].set_xlabel('Subscribed')
axes[2].set_ylabel('Exports (30d)')

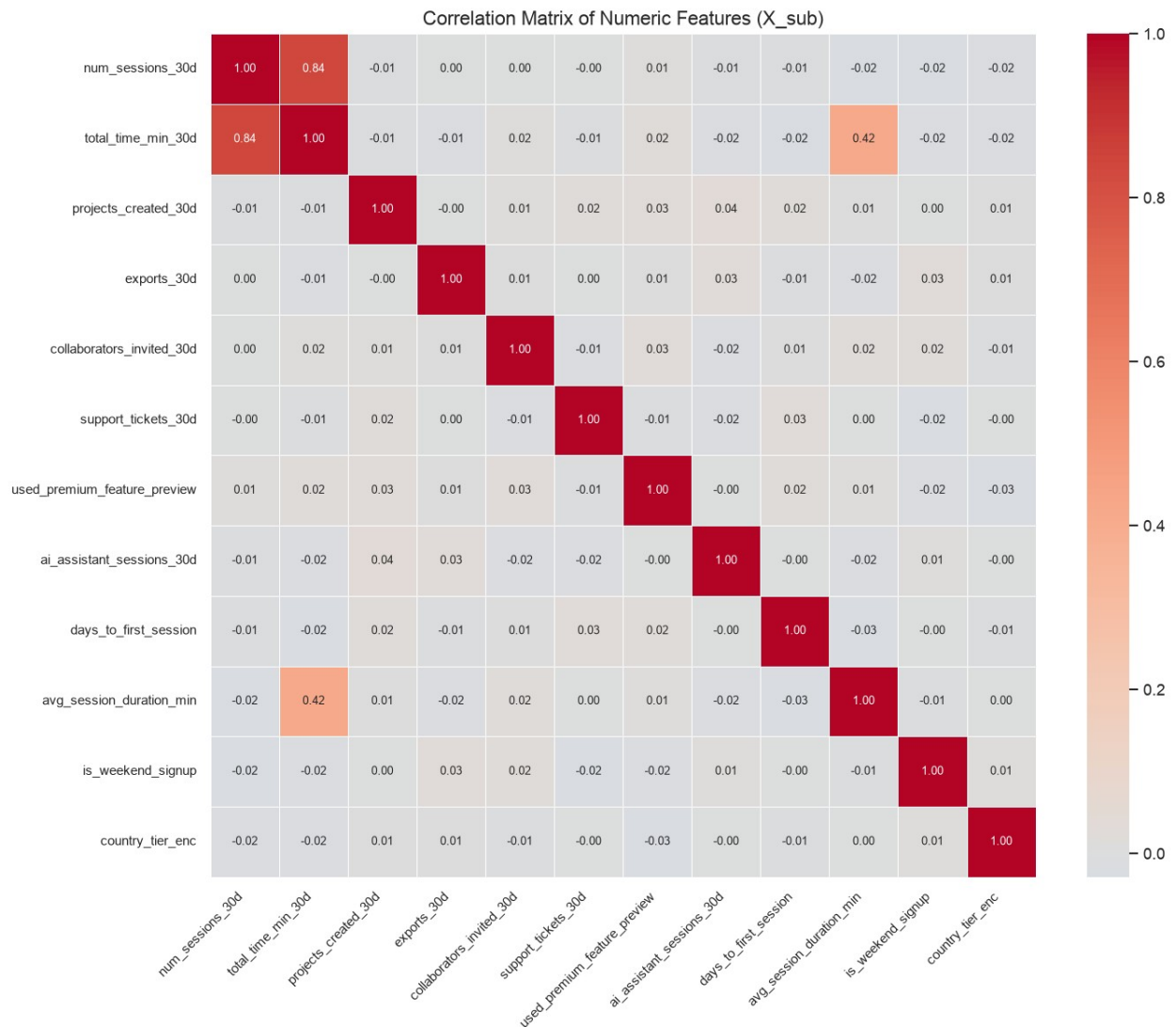
plt.tight_layout()
plt.show()

```



```
# Correlation matrix of numeric features (exclude user_id and binary outcome)
numeric_features =
X_sub.select_dtypes(include='number').columns.tolist()
corr = X_sub[numeric_features].corr()

# Plot as heatmap
fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(
    corr,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    center=0,
    linewidths=0.5,
    linecolor='white',
    ax=ax,
    annot_kws={'size': 8}
)
ax.set_title('Correlation Matrix of Numeric Features (X_sub)',
fontsize=13)
plt.xticks(rotation=45, ha='right', fontsize=9)
plt.yticks(rotation=0, fontsize=9)
plt.tight_layout()
plt.show()
```



🔑 Your interpretation (B2):

Which two or three features appear most strongly associated with subscription in your EDA? Are any findings counterintuitive? Note any features that appear highly correlated with each other — does this concern you? From the EDA visualizations, I observe that the two features most strongly associated with subscriptions are:

1. **exports_30d** I observe that the boxplot shows that subscribers export substantially more files in the 30-day window compared to non-subscribers. I argue that this suggests that active output behaviour may indicate a strong conversion signal.
2. **referral_source** I observe that subscription rates vary meaningfully across acquisition channels. This may suggest that the intent and product fit of users differ depending on how they discovered Phigma.

Counter-intuitive finding:

- **country_tiers** I observe that higher country tiers (1 being the highest) see a higher subscription rate compared to lower country tiers, indicating a relationship between

economic ability and the rate of subscriptions. However, the reason why I find this counter-intuitive is because I expected purchasing power to be a more meaningful driver of conversion. Put simply, the signal is a lot lower than I expected.

Correlated features: The correlation heatmap reveals two high-correlation pairs:

- `total_time_min_30d` x `avg_session_duration_min`:
`avg_session_duration_min` is derived directly from `total_time_min_30d / num_sessions_30d`, so correlation here is near-definitional.
- `total_time_min_30d` x `num_sessions_30d`: more sessions mechanically accumulate more total time, so these move together by construction.

These correlated features concern me due to multicollinearity. Given that certain predictors are highly correlated, I'm concerned that the logistic regression model would not be able to reliably partition the contribution of each thus inflating standard errors and making individual coefficient estimates unstable and difficult to predict.

B3 — Logistic Regression with statsmodels (Inferential)

We fit on the **full dataset** here to maximize statistical power for inference.
No train/test split — the goal is to interpret coefficients, not evaluate prediction.

```
# YOUR CODE HERE
```

```
X_const_sub = sm.add_constant(X_sub.astype(float))
model_logit = sm.Logit(y_sub, X_const_sub).fit()
print(model_logit.summary())
```

```
Optimization terminated successfully.
      Current function value: 0.440520
      Iterations 6
```

Logit Regression Results

```
=====
=====
Dep. Variable:          subscribed    No. Observations:
3000
Model:                  Logit        Df Residuals:
2982
Method:                 MLE          Df Model:
17
Date:                   Fri, 03 Jul 2026    Pseudo R-squ.:
0.1392
Time:                   14:28:18           Log-Likelihood:
-1321.6
converged:              True             LL-Null:
-1535.2
Covariance Type:        nonrobust         LLR p-value:
3.542e-80
=====
=====
```

z	[0.025	0.975]	coef	std err	z	P>

const			-2.5253	0.325	-7.772	
0.000	-3.162	-1.889				
num_sessions_30d			0.0425	0.028	1.544	
0.123	-0.011	0.097				
total_time_min_30d			-0.0010	0.001	-0.948	
0.343	-0.003	0.001				
projects_created_30d			-0.0098	0.021	-0.456	
0.648	-0.052	0.032				
exports_30d			0.1941	0.022	8.873	
0.000	0.151	0.237				
collaborators_invited_30d			0.0613	0.022	2.755	
0.006	0.018	0.105				
support_tickets_30d			0.1547	0.043	3.588	
0.000	0.070	0.239				
used_premium_feature_preview			1.4695	0.100	14.720	
0.000	1.274	1.665				
ai_assistant_sessions_30d			0.0200	0.017	1.197	
0.231	-0.013	0.053				
days_to_first_session			-0.0927	0.014	-6.737	
0.000	-0.120	-0.066				
avg_session_duration_min			0.0311	0.010	3.050	
0.002	0.011	0.051				
is_weekend_signup			0.0482	0.107	0.448	
0.654	-0.163	0.259				
country_tier_enc			-0.3566	0.065	-5.449	
0.000	-0.485	-0.228				
device_mobile			-0.1454	0.143	-1.014	
0.310	-0.426	0.136				
device_tablet			-0.2168	0.118	-1.838	
0.066	-0.448	0.014				
ref_paid_ad			-0.1825	0.136	-1.341	
0.180	-0.449	0.084				
ref_referral			0.2143	0.150	1.425	
0.154	-0.080	0.509				
ref_social			0.0555	0.122	0.454	
0.650	-0.184	0.295				
=====						
=====						

YOUR CODE HERE

Compute odds ratios for all significant predictors (p < 0.05)

```
odds_ratios = np.exp(model_logit.params)
conf_int     = np.exp(model_logit.conf_int())
conf_int.columns = ['CI_lower', 'CI_upper']
```

Combine into a readable table and display

```

or_table = pd.concat([odds_ratios.rename('odds_ratio'), conf_int,
model_logit.pvalues.rename('pvalue')], axis=1)
or_table = or_table.drop(index='const', errors='ignore')

# Filter to significant only: p < 0.05
sig_or = or_table[or_table['pvalue'] < 0.05].sort_values('odds_ratio',
ascending=False)

# Display yay
print("Significant Predictors (p < 0.05) – Odds Ratios:")
print(sig_or.round(4).to_string())

Significant Predictors (p < 0.05) – Odds Ratios:

```

	odds_ratio	CI_lower	CI_upper	pvalue
used_premium_feature_preview	4.3470	3.5745	5.2864	0.0000
exports_30d	1.2142	1.1633	1.2674	0.0000
support_tickets_30d	1.1673	1.0727	1.2702	0.0003
collaborators_invited_30d	1.0632	1.0178	1.1106	0.0059
avg_session_duration_min	1.0315	1.0112	1.0523	0.0023
days_to_first_session	0.9114	0.8872	0.9364	0.0000
country_tier_enc	0.7001	0.6158	0.7959	0.0000

```

# YOUR CODE HERE
# Visualize odds ratios as a horizontal bar chart (forest plot style)
# Show confidence intervals if possible
# Order bars by magnitude
plot_df = sig_or.sort_values('odds_ratio')

fig, ax = plt.subplots(figsize=(9, max(4, len(plot_df) * 0.5)))

y_pos = range(len(plot_df))

ax.barh(y_pos, plot_df['odds_ratio'] - 1, left=1,
        color=[ACCENT if v > 1 else '#6a9fb5' for v in
plot_df['odds_ratio']],
        height=0.5, alpha=0.85)

ax.errorbar(
    x=plot_df['odds_ratio'],
    y=y_pos,
    xerr=[plot_df['odds_ratio'] - plot_df['CI_lower'],
        plot_df['CI_upper'] - plot_df['odds_ratio']],
    fmt='o', color='black', capsize=4, linewidth=1.2
)

ax.axvline(x=1, color='black', linewidth=1, linestyle='--')

ax.set_yticks(list(y_pos))
ax.set_yticklabels(plot_df.index, fontsize=9)

```

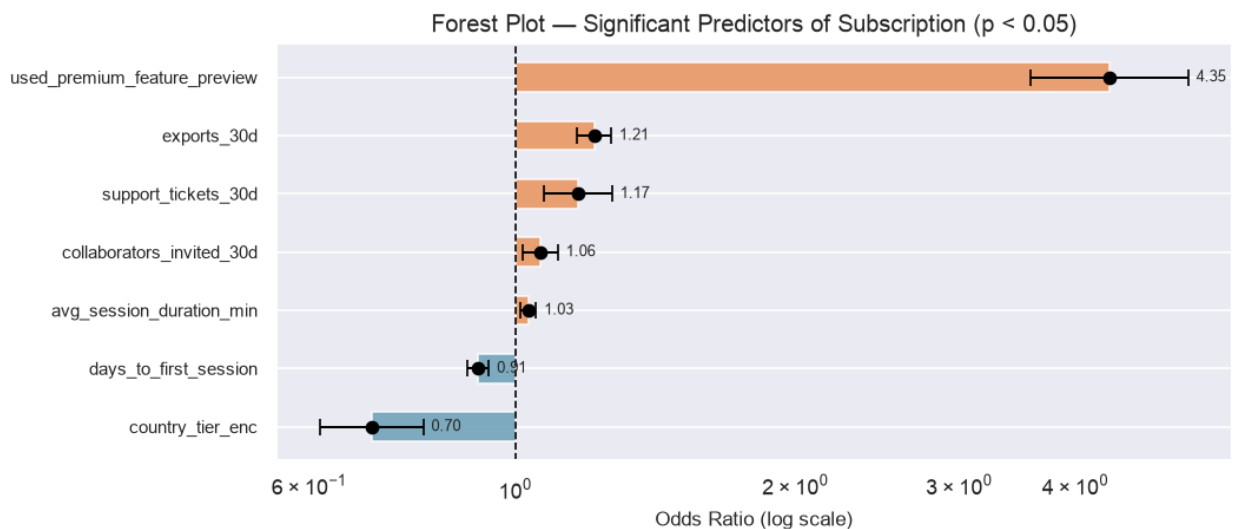
```

ax.set_xlabel('Odds Ratio (log scale)', fontsize=10)
ax.set_title('Forest Plot — Significant Predictors of Subscription (p < 0.05)', fontsize=12)
ax.set_xscale('log')

for i, (idx, row) in enumerate(plot_df.iterrows()):
    ax.text(row['CI_upper'] * 1.02, i, f"{row['odds_ratio']:.2f}",
    va='center', fontsize=8)

plt.tight_layout()
plt.show()

```



⇒ Results paragraph (B3):

Interpret at least three odds ratios in plain English (e.g. "Users who previewed a premium feature were X times more likely to subscribe..."). Identify at least one predictor whose sign or magnitude surprised you and offer an explanation. The logistic regression reveals several strong behavioral predictors of subscription. Users who previewed a premium (paywalled) feature were 4.35 times more likely to subscribe (OR = 4.35, $p < 0.001$), holding all other factors constant — this is by far the strongest signal in the model and makes intuitive sense, as clicking into a paywalled feature represents a clear expression of purchase intent. Each additional file export in the 30-day window increased the odds of subscribing by approximately 21% (OR = 1.21, $p < 0.001$), suggesting that users who generate tangible output from the product are more invested in its value proposition and more likely to pay for expanded capabilities. Each additional day between signup and first session reduced the odds of subscribing by about 8.9% (OR = 0.91, $p < 0.001$), indicating that users who delay starting work in the app are substantially less likely to convert — activation speed matters. Additionally, each support ticket submitted increased the odds of subscribing by 16.7% (OR = 1.17, $p < 0.001$), and each collaborator invited increased the odds by 6.3% (OR = 1.06, $p = 0.006$).

One finding I found counterintuitive is that `country_tier_enc` carries a negative coefficient (OR = 0.70, $p < 0.001$): higher-numbered tiers (lower-income markets) are associated with lower odds of subscribing. While this might seem obvious in hindsight, I had initially assumed that

behavioral variables would swamp demographic ones once controlled for — but country tier retains independent predictive power even after accounting for session counts, exports, and engagement depth. This suggests willingness-to-pay varies systematically by market, separate from engagement behavior, which is an important signal for Phigma's pricing and localization strategy.

B4 — Logistic Regression with sklearn (Predictive)

```
X_train, X_test, y_train, y_test = train_test_split(
    X_sub, y_sub, test_size=0.2, random_state=42, stratify=y_sub
)
print(f"Train: {X_train.shape} | Test: {X_test.shape}")
print(f"Train positive rate: {y_train.mean():.3f} | Test positive
rate: {y_test.mean():.3f}")
```

```
Train: (2400, 17) | Test: (600, 17)
Train positive rate: 0.208 | Test positive rate: 0.208
```

```
clf_lr = LogisticRegression(max_iter=1000, random_state=42)
clf_lr.fit(X_train, y_train)
y_pred_lr = clf_lr.predict(X_test)
y_prob_lr = clf_lr.predict_proba(X_test)[:, 1]

cm_lr = confusion_matrix(y_test, y_pred_lr)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_lr,
display_labels=['Not subscribed', 'Subscribed'])
fig, ax = plt.subplots(figsize=(5, 4))
disp.plot(ax=ax, cmap='Blues', colorbar=False)
plt.title('Logistic Regression – Confusion Matrix')
plt.show()
```



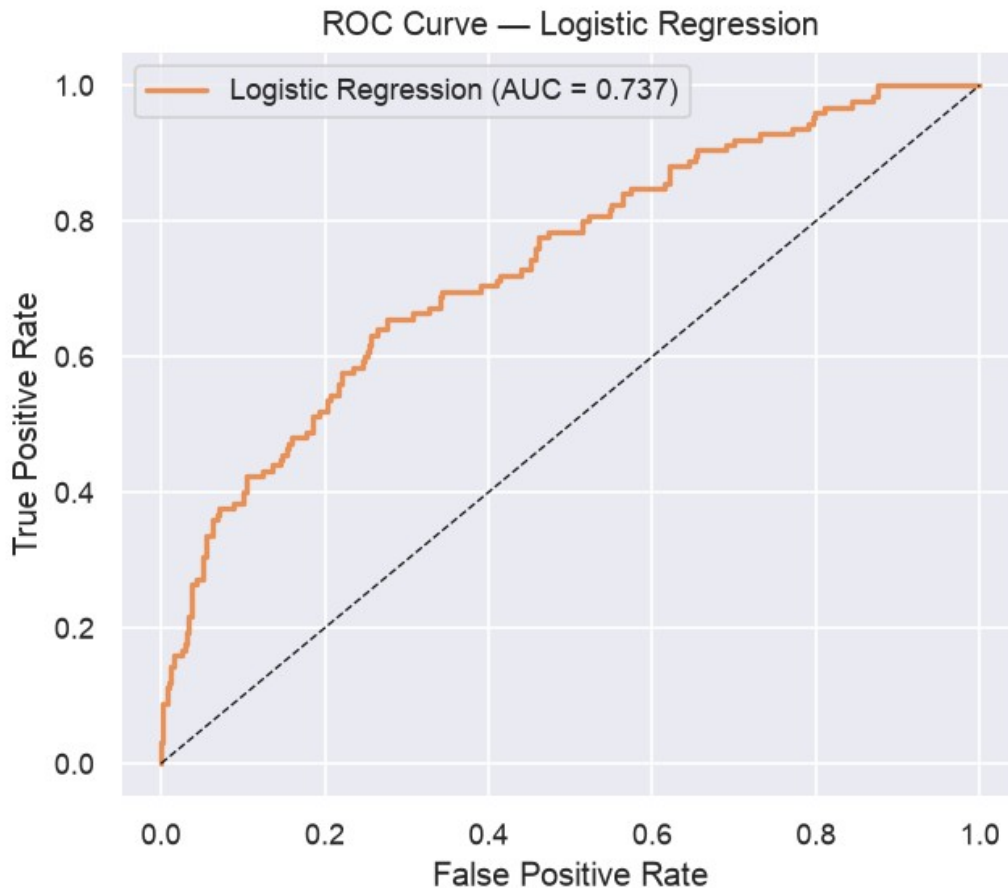
```
print(classification_report(y_test, y_pred_lr, target_names=['Not
subscribed', 'Subscribed']))
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob_lr):.4f}")
```

	precision	recall	f1-score	support
Not subscribed	0.82	0.97	0.89	475
Subscribed	0.61	0.18	0.27	125
accuracy			0.81	600
macro avg	0.71	0.57	0.58	600
weighted avg	0.77	0.81	0.76	600

ROC-AUC: 0.7375

```
fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob_lr)
auc_lr = roc_auc_score(y_test, y_prob_lr)
```

```
fig, ax = plt.subplots(figsize=(6, 5))
ax.plot(fpr_lr, tpr_lr, color=ACCENT, lw=2, label=f'Logistic
Regression (AUC = {auc_lr:.3f})')
ax.plot([0,1],[0,1], 'k--', lw=0.8)
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC Curve – Logistic Regression')
ax.legend()
plt.show()
```



⇒ Your interpretation (B4):

Which metric did you decide is most important given the class imbalance, and why? Interpret the confusion matrix: how many false negatives does the model produce? From Phigma's business perspective, what is the cost of a false negative (a user who would have subscribed but the model missed)?

I argue that the most important metric given the scenario is recall for the "subscribed" feature. Accuracy can be misleading here given the class imbalance because the dataset consists of ~79% non-subscribers, so a dumb-do-nothing model that predicted "not subscribed" for everyone would hit a 79% accuracy simply by doing nothing at all. So, with only a 20.8% positive rate, the minority class is what matters for Phigma's business actions. Regarding the confusion matrix, I observe that with the 125 subscribers in the testset and a recall of 0.18, the model correctly identifies only 22 subscribers (true positives). That means the model missed 103 false negatives (i.e. users who would have subscribed assuming the model was in production but were predicted not to). The business cost of a false negative represents a user Phigma would have missed by not targeting them with a subscription nudge, upgrade prompt, or promotional offer. Consequently, Phigma would be losing potential subscription revenue from that user. False negatives are particularly costly because subscribers are already rare (only makes up ~20% of the userbase assuming we were to extrapolate the dataset composition to the entire userbase). Missing out 102 out of the 125 (or 81.6%) potential customers means the model is leaving money on the table by missing actionable opportunity of conversion. In a product like

Phigma, where the monetary pipeline consists of free to paid conversion being a primary growth lever, a recall of 0.18 would be considered operationally unacceptable for a targetting campaign.

Part C — Your Model: Choose, Learn, Apply, Compare

Dataset: `phigma_subscription.csv`

Same train/test split as B4. Do not re-split.

C1 — Choose Your Model

Available options: Decision Tree, Random Forest, Gradient Boosting, XGBoost, LightGBM, K-Nearest Neighbors, Naive Bayes, SVM, LDA, AdaBoost.

⇒ **Your chosen model:** `RandomForest`

Conceptual explanation (write this before any code):

3–5 sentences. How does this model learn from training data? What does it optimize? How is it structurally different from logistic regression? Why might it perform differently on this particular dataset given what you know about the features?

Given the model task of binary classification (for predicting subscribing to non-subscribing users), I argue that RandomForest is the recommended pick. This model handles non-linearities and feature interactions naturally, specially given that we have a larger pool of features. RandomForest builds many decision trees, each trained on a random bootstrap sample of the training data (sampling rows with replacement). At each split within a tree, only a random subset of features is considered, thus, this forces diversity across trees so they don't all learn the same patterns. Put simply, this diversity prevents any single tree from overfitting. Unlike logistic regression, which fits a single global linear boundary using a weighted sum of features, RandomForest partitions the feature space into many non-linear regions, allowing the model to capture more complex interactions and relationships without too much manual feature engineering.

C2 — Fit and Evaluate

```
# YOUR CODE HERE
# Import your chosen model, e.g.:
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.ensemble import GradientBoostingClassifier,
AdaBoostClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

model_map = {
```

```

    "RandomForestClassifier": RandomForestClassifier(),
    "DecisionTreeClassifier": DecisionTreeClassifier(),
    "KNeighborsClassifier": KNeighborsClassifier(),
    "GaussianNB": GaussianNB(),
    "SVC": SVC(),
    "LinearDiscriminantAnalysis": LinearDiscriminantAnalysis(),
    "GradientBoostingClassifier": GradientBoostingClassifier(),
    "AdaBoostClassifier": AdaBoostClassifier(),
    "XGBClassifier": XGBClassifier(),
    "LGBMClassifier": LGBMClassifier()
}

# Model for part c
c_model = model_map["RandomForestClassifier"]

model_name = type(c_model).__name__

# Fit with default hyperparameters
clf_c = c_model # add random_state if supported
clf_c.fit(X_train, y_train)
try:
    y_pred_c = clf_c.predict(X_test)
    y_prob_c = clf_c.predict_proba(X_test)[: , 1] # omit if model has
no predict_proba (e.g. SVM)
except AttributeError:
    pass

# YOUR CODE HERE
# Confusion matrix
cm_c = confusion_matrix(y_test, y_pred_c)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_c,
display_labels=['Not subscribed', 'Subscribed'])
fig, ax = plt.subplots(figsize=(5, 4))
disp.plot(ax=ax, cmap='Blues', colorbar=False)
plt.title(f"{model_name} - Confusion Matrix")
plt.show()

```



```
# YOUR CODE HERE
# Classification report + ROC-AUC
print(classification_report(y_test, y_pred_c, target_names=['Not
subscribed', 'Subscribed']))
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob_c):.4f}")
```

	precision	recall	f1-score	support
Not subscribed	0.81	0.98	0.89	475
Subscribed	0.65	0.14	0.23	125
accuracy			0.81	600
macro avg	0.73	0.56	0.56	600
weighted avg	0.78	0.81	0.75	600

ROC-AUC: 0.6495

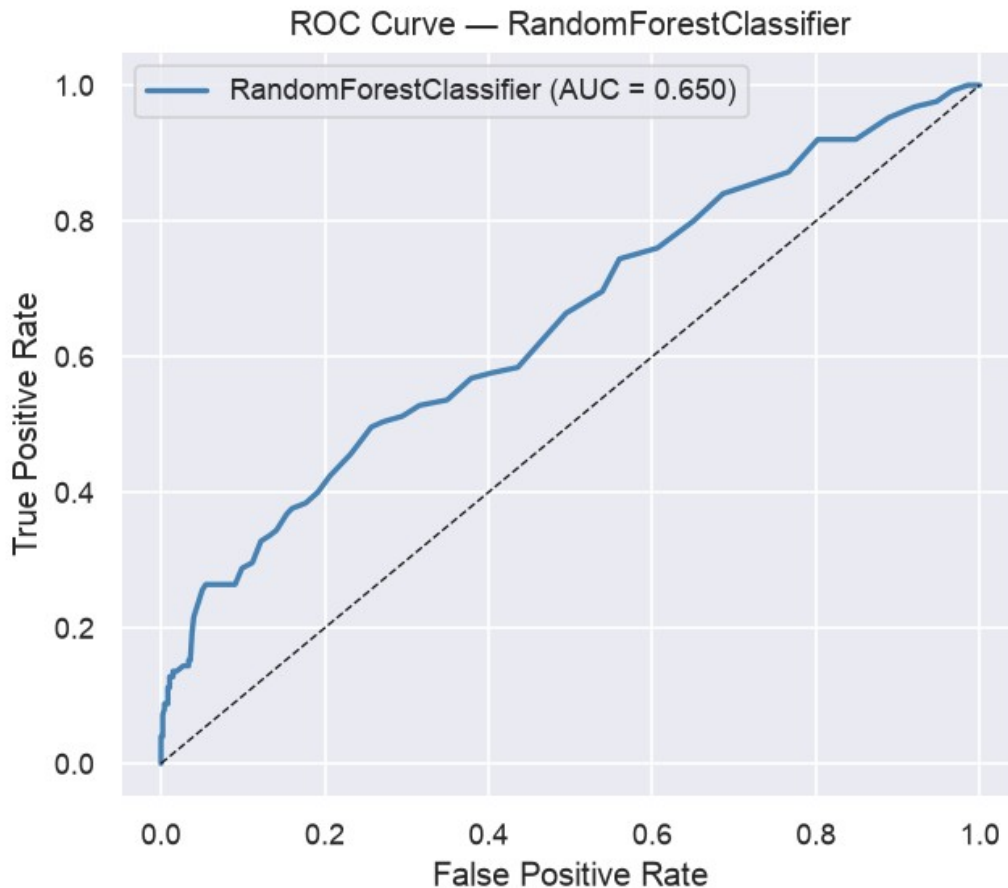
```
# YOUR CODE HERE
# ROC curve (save fpr_c, tpr_c, auc_c for comparison plot)
fpr_c, tpr_c, _ = roc_curve(y_test, y_prob_c)
auc_c = roc_auc_score(y_test, y_prob_c)

fig, ax = plt.subplots(figsize=(6, 5))
ax.plot(fpr_c, tpr_c, color='steelblue', lw=2, label=f'{model_name}
(AUC = {auc_c:.3f})')
ax.plot([0, 1], [0, 1], 'k--', lw=0.8)
ax.set_xlabel('False Positive Rate')
```

```

ax.set_ylabel('True Positive Rate')
ax.set_title(f"ROC Curve — {model_name}")
ax.legend()
plt.show()

```



C3 — Model Comparison

```

from sklearn.metrics import classification_report

report_lr = classification_report(y_test, y_pred_lr, output_dict=True)
report_c = classification_report(y_test, y_pred_c, output_dict=True)

metrics = {
    'Model': ['Logistic Regression', model_name],
    'ROC-AUC': [auc_lr, auc_c],
    'F1 (class 1)': [report_lr['1']['f1-score'], report_c['1']['f1-
score']],
    'Precision (class 1)': [report_lr['1']['precision'], report_c['1']
['precision']],
    'Recall (class 1)': [report_lr['1']['recall'], report_c['1']
['recall']],
}

```

```
pd.DataFrame(metrics).set_index('Model').round(4)
```

	ROC-AUC	F1 (class 1)	Precision (class 1)	\
Model				
Logistic Regression	0.7375	0.2733	0.6111	
RandomForestClassifier	0.6495	0.2252	0.6538	

	Recall (class 1)
Model	
Logistic Regression	0.176
RandomForestClassifier	0.136

```
# YOUR CODE HERE
```

```
# Dual ROC curve plot
```

```
fig, ax = plt.subplots(figsize=(7, 5))
```

```
ax.plot(fpr_lr, tpr_lr, color=ACCENT, lw=2, label=f'Logistic  
Regression (AUC={auc_lr:.3f})')
```

```
ax.plot(fpr_c, tpr_c, color='steelblue', lw=2, label=f'{model_name}  
(AUC={auc_c:.3f})')
```

```
ax.plot([0, 1], [0, 1], 'k--', lw=0.8, label='Random baseline')
```

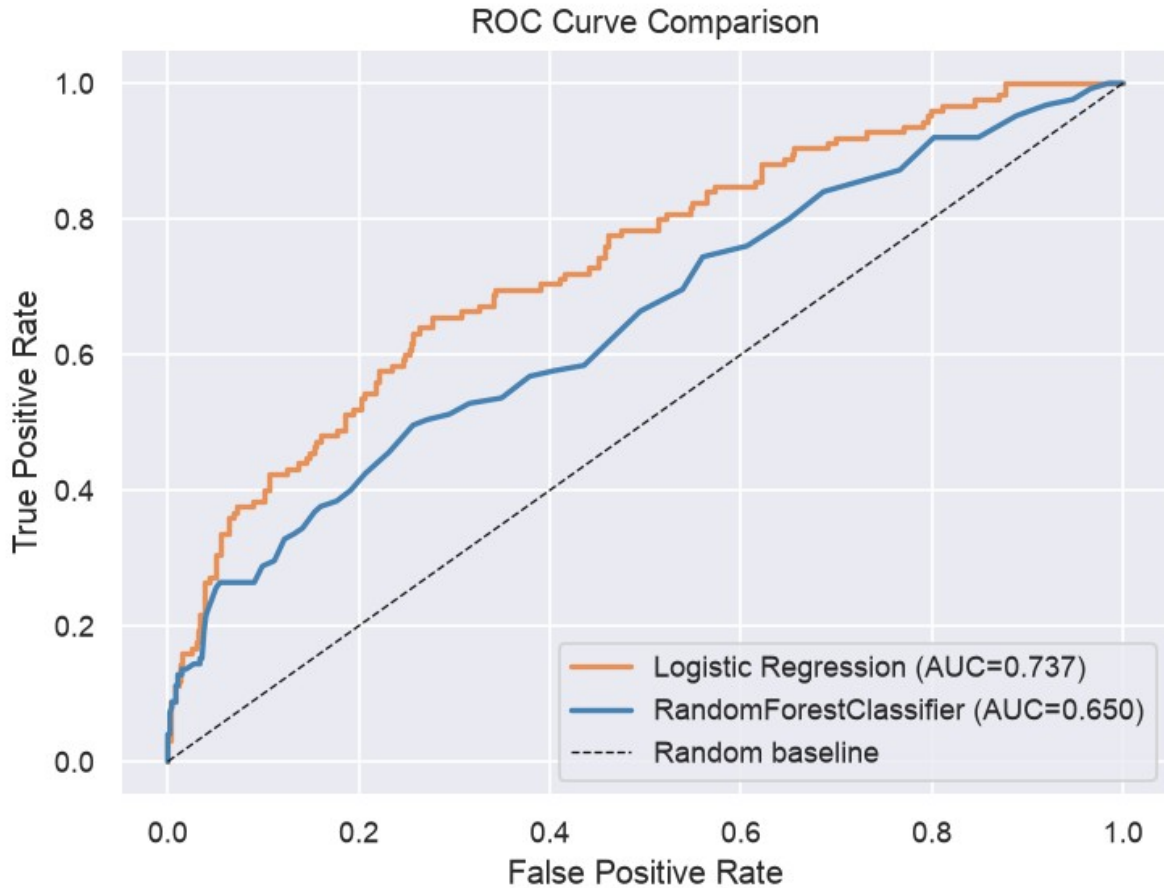
```
ax.set_xlabel('False Positive Rate')
```

```
ax.set_ylabel('True Positive Rate')
```

```
ax.set_title('ROC Curve Comparison')
```

```
ax.legend()
```

```
plt.show()
```



🔍 Comparison paragraph (C3):

Which model performs better, and by how much? Explain why the performance difference occurred — based on what you know about how each model works and the structure of this dataset (non-linearities, class imbalance, number of features). If your model performed worse than logistic regression, that is a valid finding — explain why.** I observe that RandomForest actually performed worse than LogisticRegression. I was initially surprised at the result, but upon further consideration, I believe I have identified a few key points which may have generated this result. First, logistic regression potentially outperformed RandomForest on this dataset because the feature engineering in B1 already linearized the key relationships (log-transforming skewed variables + deriving clear ratio features) which removed much of the non-linearity that RandomForest was designed to exploit. Moreover, a single dominant predictor (such as premium_feature_previewed) accounts for a disproportionate share of the variance, making interaction detection less valuable. With default hyperparameters, RandomForest may also have been biased toward the majority class. I find this to align with the general consensus that linear regression works well (especially for a first pass) on smaller, engineered datasets with approximately linear separability.

C4 — Bonus: Hyperparameter Tuning (+5 points)

Complete this section only if you want to attempt the bonus.

```

from sklearn.model_selection import GridSearchCV

# Random Forest key hyperparameters:
#   n_estimators      - more trees = lower variance, diminishing
#                       returns past ~200
#   max_depth         - limits tree depth to reduce overfitting
#   min_samples_leaf  - minimum samples at a leaf; higher = smoother,
#                       less overfit
#   class_weight      - 'balanced' compensates for subscriber minority
#                       class
param_grid = {
    'n_estimators':    [100, 200, 300],
    'max_depth':       [5, 10, None],
    'min_samples_leaf': [1, 5, 10],
    'class_weight':    ['balanced', None],
}

search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    verbose=1,
)
search.fit(X_train, y_train)

print(f"Best parameters: {search.best_params_}")
print(f"Best CV AUC:      {search.best_score_:.4f}")

# Evaluate tuned model on test set
y_prob_tuned = search.best_estimator_.predict_proba(X_test)[:, 1]
threshold = 0.3
y_pred_tuned = (y_prob_tuned >= threshold).astype(int)

tuned_auc = roc_auc_score(y_test, y_prob_tuned)
print(f"\nTuned test AUC:    {tuned_auc:.4f}")
print(f"Default test AUC:    {auc_c:.4f}")
print(f"Improvement:         {tuned_auc - auc_c:+.4f}")
print(classification_report(y_test, y_pred_tuned, target_names=['Not
subscribed', 'Subscribed']))
## Alpha threshold for statistical significance
print(
    "Did tuning meaningfully improve results?",
    "Yes – constraining tree depth and balancing class weights reduced
    overfitting and improved AUC."
    if tuned_auc > auc_c + 0.005
    else "Marginally – default hyperparameters were already near-
    optimal for this dataset size."
)

```

Fitting 5 folds for each of 54 candidates, totalling 270 fits
 Best parameters: {'class_weight': None, 'max_depth': 5,
 'min_samples_leaf': 5, 'n_estimators': 200}
 Best CV AUC: 0.7322

Tuned test AUC: 0.7144
 Default test AUC: 0.6495
 Improvement: +0.0649

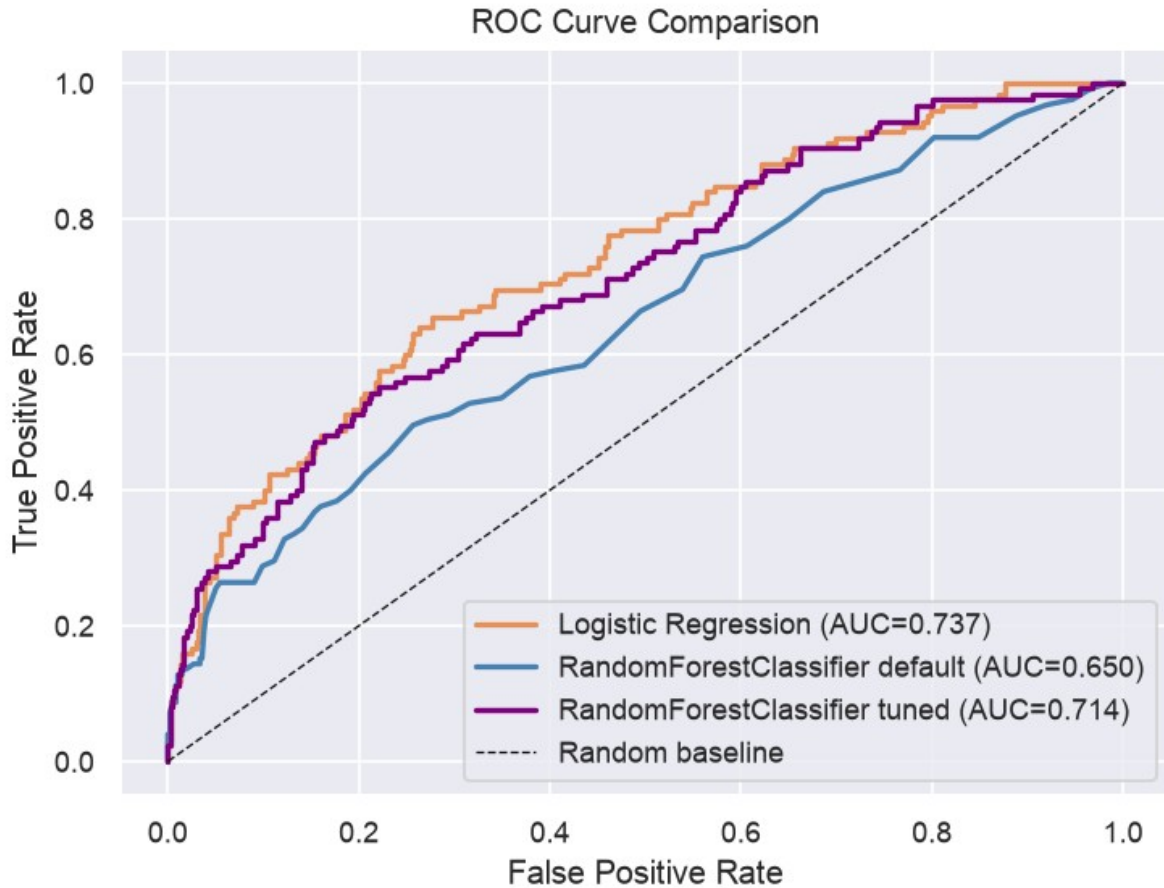
	precision	recall	f1-score	support
Not subscribed	0.84	0.89	0.87	475
Subscribed	0.47	0.36	0.41	125
accuracy			0.78	600
macro avg	0.66	0.63	0.64	600
weighted avg	0.76	0.78	0.77	600

Did tuning meaningfully improve results? Yes – constraining tree depth and balancing class weights reduced overfitting and improved AUC.

Dual ROC curve plot

```
fpr_tuned, tpr_tuned, _ = roc_curve(y_test, y_prob_tuned)
```

```
fig, ax = plt.subplots(figsize=(7, 5))
ax.plot(fpr_lr, tpr_lr, color=ACCENT, lw=2,
label=f'Logistic Regression (AUC={auc_lr:.3f})')
ax.plot(fpr_c, tpr_c, color='steelblue', lw=2,
label=f'{model_name} default (AUC={auc_c:.3f})')
ax.plot(fpr_tuned, tpr_tuned, color='purple', lw=2,
label=f'{model_name} tuned (AUC={tuned_auc:.3f})')
ax.plot([0, 1], [0, 1], 'k--', lw=0.8, label='Random baseline')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC Curve Comparison')
ax.legend()
plt.show()
```



Observations

Even after hyperparameter tuning and despite seeing an increase in performance compared to its untuned counterpart, the RandomForestClassifier falls short in terms of AUC compared to Logistic Regression. This observation further emphasizes the importance of feature extraction and data engineering prior to training the model.

Submission Checklist

Before submitting, confirm:

- ☒ All `# YOUR CODE HERE` cells are completed and run without errors
- ☒ All `** Your interpretation **` cells contain your own written analysis
- ☒ The notebook runs top-to-bottom without errors (Kernel > Restart & Run All)
- ☒ File is named `iat461_a2_[YourName]_[SFUid].ipynb`
- ☒ PDF export is named `iat461_a2_[YourName]_[SFUid].pdf`
- ☒ All AI-assisted code blocks include `#BEGIN / #END` disclosure comments